

Librería "El Mundo de Sofía"

Henry Morales

1. Introducción

Este documento explica el diseño de la base de datos para la librería "El Mundo de Sofía". La tienda necesita controlar su inventario de libros, registrar autores, clientes, pedidos y las transacciones de pago. El objetivo es tener un modelo claro, sin datos repetidos, que facilite las búsquedas por título, autor, categoría e ISBN.

Para resolverlo decidí usar el modelo relacional con MySQL, aprovechando claves primarias, foráneas e índices que garanticen la integridad de los datos. El diagrama UML E-R que acompaña a este documento muestra cómo se conectan las entidades y las cardinalidades de cada relación.

2. Justificación del diseño

Al analizar los requerimientos me di cuenta de que había que separar la información en tablas pequeñas y especializadas. Esto evita la redundancia y permite que los cambios se hagan en un solo lugar. A continuación explico las decisiones más importantes:

- Tablas maestras separadas: los autores, editoriales y categorías están en tablas propias para no repetir sus nombres dentro de cada libro.
- Tabla intermedia libro_autor: un libro puede tener varios autores y un autor puede haber escrito varios libros, por eso se necesita una tabla N:M.
- Tabla detalle_pedido: como un pedido puede incluir varios libros, separé el pedido de sus líneas de detalle. Cada línea guarda la cantidad y el precio unitario en ese momento.
- Tabla direccion aparte del cliente: aunque el requerimiento pide dirección, preferí normalizarla en su propia tabla porque un cliente puede tener más de una dirección (casa, trabajo, etc.).
- Transacción vinculada 1 a 1 con pedido: como cada pedido genera una sola transacción, la tabla transaccion tiene una clave foránea única hacia pedido.

También decidí usar el motor InnoDB para todas las tablas, ya que soporta claves foráneas y transacciones, lo cual es importante para mantener la integridad cuando se registran pedidos y pagos.

3. Entidades y atributos

A continuación describo cada tabla y los campos que la componen:

Tabla: autor

- id_autor (PK): identificador único, autoincremental.
- nombre, apellido: datos personales del autor, obligatorios.
- fecha_nacimiento: fecha opcional.
- nacionalidad: país de origen opcional.

Tabla: editorial

- id_editorial (PK): identificador único, autoincremental.
- nombre: nombre de la editorial, obligatorio y único.
- pais: país de la editorial, opcional.

Tabla: categoria

- id_categoria (PK): identificador único, autoincremental.
- nombre: nombre de la categoría, obligatorio y único.
- descripcion: explicación opcional de la categoría.

Tabla: libro

- id_libro (PK): identificador único, autoincremental.
- isbn (UQ): código ISBN de 13 caracteres, único para evitar duplicados.
- titulo: título del libro, obligatorio.
- id_editorial (FK): editorial que lo publicó.
- id_categoria (FK): categoría a la que pertenece.
- fecha_publicacion: fecha opcional.
- precio: precio de venta, obligatorio.
- stock: cantidad disponible, con valor por defecto 0.

Tabla: libro_autor

- id_libro (PK, FK) y id_autor (PK, FK): claves compuestas que vinculan libro y autor.
- orden: permite ordenar a los autores de un mismo libro.

Tabla: cliente

- id_cliente (PK): identificador único, autoincremental.
- nombre, apellido: obligatorios.
- email (UQ): correo único para evitar registros duplicados.
- telefono: opcional.
- fecha_registro: fecha y hora del registro.

Tabla: direccion

- id_direccion (PK): identificador único.
- id_cliente (FK): cliente al que pertenece.
- calle, ciudad, pais: obligatorios.
- estado, codigo_postal: opcionales.
- es_principal: indica si es la dirección principal del cliente.

Tabla: pedido

- id_pedido (PK): identificador único.
- id_cliente (FK): cliente que hizo el pedido.
- id_direccion (FK): dirección de entrega, puede ser nula.

- fecha_compra: momento del pedido.
- estado: ENUM(pendiente, procesado, completado, cancelado).

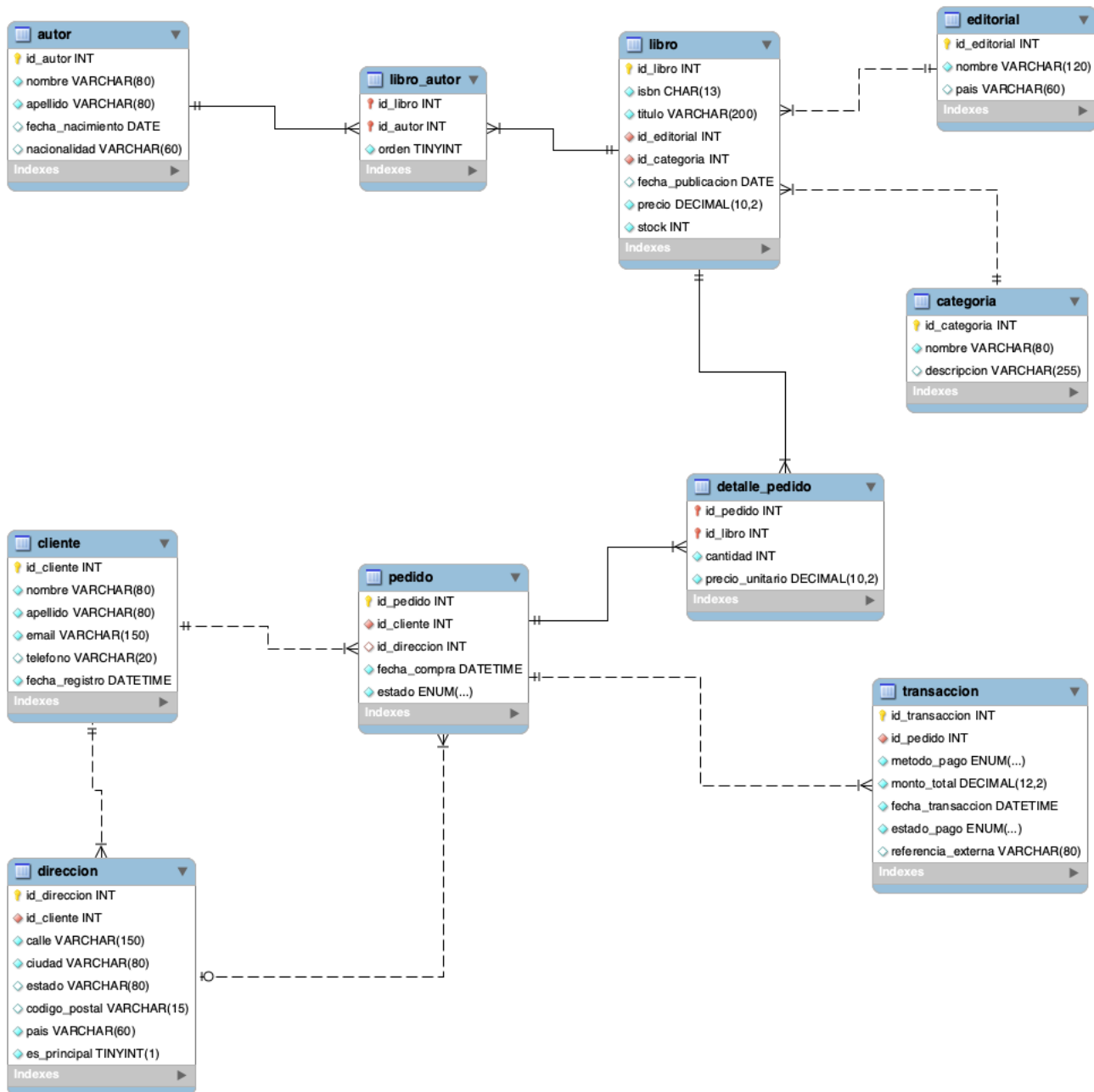
Tabla: detalle_pedido

- id_pedido (PK, FK) e id_libro (PK, FK): claves compuestas.
- cantidad: unidades del libro solicitadas.
- precio_unitario: precio al momento de la compra.

Tabla: transaccion

- id_transaccion (PK): identificador único.
- id_pedido (FK, UQ): vinculado uno a uno con el pedido.
- metodo_pago: ENUM(tarjeta_credito, tarjeta_debito, paypal, transferencia, efectivo).
- monto_total: total pagado.
- fecha_transaccion: momento del pago.
- estado_pago: ENUM(authorizada, rechazada, reembolsada).
- referencia_externa: código opcional del procesador de pagos.

4. Diagrama UML E-R



5. Relaciones UML y cardinalidad

autor ↔ libro (N:M)

Un autor puede escribir muchos libros y un libro puede tener varios autores. La relación se resuelve con la tabla intermedia libro_autor.

editorial → libro (1:N)

Una editorial publica muchos libros, pero cada libro pertenece a una sola editorial.

categoria → libro (1:N)

Una categoría agrupa muchos libros, pero un libro está en una sola categoría.

cliente → direccion (1:N)

Un cliente puede tener varias direcciones; cada dirección pertenece a un solo cliente.

cliente → pedido (1:N)

Un cliente puede realizar muchos pedidos; cada pedido es de un solo cliente.

direccion → pedido (1:N, opcional)

Un pedido puede entregarse en una dirección; la dirección es opcional.

pedido → detalle_pedido (1:N)

Un pedido tiene al menos una línea de detalle; cada línea pertenece a un solo pedido.

libro → detalle_pedido (1:N)

Un libro puede aparecer en muchos detalles de pedido; cada línea se refiere a un solo libro.

pedido ↔ transaccion (1:1)

Cada pedido genera exactamente una transacción; cada transacción corresponde a un solo pedido.

6. Restricciones y validaciones

Se aplicaron las siguientes restricciones para mantener la calidad de los datos:

- Claves primarias (PRIMARY KEY): cada tabla tiene un identificador único, en la mayoría de los casos autoincremental.
- Claves foráneas (FOREIGN KEY): garantizan que las relaciones sean válidas y evitan que se elimine una editorial o categoría si tiene libros asociados.
- Unicidad (UNIQUE): el ISBN de los libros y el email de los clientes deben ser únicos. También la combinación nombre-apellido de autor y el nombre de editorial y categoría.
- Índices: se crearon índices en los campos de búsqueda frecuente (título del libro, cliente del pedido, libros en detalle, etc.).
- Valores por defecto: el stock inicia en 0, el estado de un pedido en "pendiente" y la fecha de registro es CURRENT_TIMESTAMP.
- CHECK: se agregaron restricciones para que precio y stock no sean negativos.
- Triggers de stock: un trigger BEFORE INSERT revisa que haya stock suficiente y un trigger AFTER INSERT descuenta el stock automáticamente de la tabla libro.

7. Cómo ejecutar el script SQL

1. Abrir MySQL Workbench o la línea de comandos de MySQL.
2. Ejecutar el archivo libreria_sofia.sql con: SOURCE /ruta/libreria_sofia.sql;
3. Verificar que las tablas se crearon correctamente con: SHOW TABLES;
4. Probar las relaciones insertando datos de prueba.

El diagrama se puede abrir con el archivo diagrama_er.puml en cualquier editor compatible con PlantUML.

Se recomienda ejecutar el script sobre MySQL 8.0 o superior para que las restricciones CHECK y los triggers funcionen correctamente.

8. Conclusión

El diseño propuesto cumple con los requerimientos del proyecto: permite gestionar libros, autores, clientes, pedidos y transacciones de forma organizada. Las relaciones están bien definidas, se evitan datos duplicados y el stock se actualiza automáticamente al registrar una compra.

Considerando que en un futuro la librería puede crecer, el modelo es fácil de ampliar; por ejemplo, se pueden agregar tablas de reseñas, descuentos o historial de precios sin afectar la estructura actual.